Escape The Chicken Island

Rendu de projet ACVL

2024-10-21

CANAGUY Vincent, LAISNE Matthew, NGONGANG Gaston, WENDELS Jérémy
Ensimag
2024-2025
2AA

Table des matières

I. Cahier des charges 4
I.1. Objectif 4
I.2. Scénario 4
I.3. Mécanismes de jeu 4
Collecte d'objets 4
Îles 4
Évolution du Chicken-Raft 4
Survie 4
Déplacement 5
Actions 5
I.4. Personnages non jouables 5
Animaux 5
Fermiers 5
I.5. Interfaces 5
Menu 5
Paramètres 5
Inventaire 5
Barre d'outils 5
Carte 5
Boîte de dialogue 5
I.6. Spécifications techniques 6
I.7. Contraintes 6
II. Document d'analyse 6
II.1. Cas d'utilisation 6
Récolter du bois 7
Récolter de la nourriture 8
Se battre contre un fermier 9
Se nourrir 10
Se soigner 11
Ouvrir un coffre au trésor 12
Finir le jeu 13
II.2. Diagramme de classes d'analyse 14
III. Document de conception 16
III.1. Architecture logicielle 16
III.2. Description de l'incrément choisit 17
III.3. Premier incrément 17

III.4. Conception détaillée 17
Diagramme de classe logicielle 17
Diagrammes de séquence 18
IV. Manuel utilisateur 20
V. Bilan sur les outils de modélisation 20
V.1. Draw.io 20
V.2. Visual Paradigm 21

I. Cahier des charges

Titre du projet : Escape The Chicken Island

Ce cahier des charges décrit le projet de jeu vidéo **Escape The Chicken Island** dans sa globalité. Il présente donc le jeu dans son ensemble et non uniquement l'incrément réalisé pour le rendu du projet.

I.1. Objectif

Développer un jeu d'aventure en 2D, avec interface graphique. Le joueur incarne un aventurier qui doit s'échapper d'un archipel. L'archipel est composé de quatre îles. Pour s'échapper, l'aventurier doit se déplacer d'île en île, sur lesquelles il doit trouver des trésors et collecter des objets afin de construire un radeau. Le jeu est accessible à tout le monde et simple à prendre en main.

I.2. Scénario

Nous sommes en 1720. Joseph, un marin courageux mais malchanceux, se retrouve naufragé après une violente tempête. Il échoue sur une île étrange nommée Chicken Island, un lieu mystérieux peuplé d'animaux et d'habitants… avec une obsession inexplicable pour les poules ! Joseph doit non seulement survivre, mais aussi comprendre pourquoi cette île entière semble être régie par les poules et leurs adorateurs.

I.3. Mécanismes de jeu

Collecte d'objets

Le personnage principal peut collecter divers objets, comme du bois, des plantes, de la nourriture, et ouvrir des trésors pour récupérer des potions, ficelles, clous, etc. Les objets permettent la survie du personnage et la construction d'un radeau pour s'échapper de l'île.

Îles

Chaque île dispose d'un trésor unique. Les îles sont reliées entre elles par une passerelle. On ne peut accéder aux îles que dans un ordre prédéfini, et l'entrée sur une île n'est possible que si le trésor de l'île précédente a été collecté. Chaque île a un taux d'apparition d'animaux différent.

Évolution du Chicken-Raft

Le bateau de Joseph est présent sur l'île de départ, mais en piteux état. C'est en ramassant divers objets en grande quantité que nous pourrons construire un nouveau radeau. Le radeau changera visuellement à chaque évolution.

Survie

Le personnage dispose d'une jauge de vie. Celle-ci diminue s'il subit des attaques d'ennemis ou si sa jauge de faim est vide. Pour augmenter la jauge de vie, il faut boire des potions présentes dans les coffres. Le personnage dispose également d'une jauge de faim ; si elle est vide, la jauge de vie diminue rapidement, entraînant sa mort. Chaque île dispose initialement d'un nombre prédéfini d'animaux, et chaque fois qu'un animal est tué, il met un certain temps à réapparaître. Le taux de réapparition des animaux varie selon les îles. Les premières îles ont

un taux d'apparition plus bas que les îles finales. Si le joueur reste trop longtemps sur une île, il n'aura plus assez de nourriture pour se nourrir et mourra, car la jauge de faim descend plus vite que l'apparition des nouveaux animaux.

Déplacement

Le joueur peut marcher, mais ne peut pas sortir des limites de la carte.

Actions

Le jeu propose un panel d'actions, comprenant :

- **Main** : permet d'ouvrir des coffres.
- **Épée** : permet de tuer des animaux et des ennemis.
- **Hache** : permet de couper des arbres pour obtenir du bois.
- **Poulet** : permet de manger du poulet pour restaurer la jauge de faim.
- **Potion** : permet de boire une potion pour restaurer la jauge de vie.

I.4. Personnages non jouables

Animaux

Il existe différents animaux, qui offrent plus ou moins de nourriture selon leur type. Par exemple, un lion fournira plus de nourriture qu'une poule. Certains animaux sont inoffensifs, tandis que d'autres peuvent attaquer le joueur.

Fermiers

Ils élèvent des animaux. Si le joueur s'approche trop de leurs animaux, ils l'attaquent. Les fermiers sont mis K.O. s'ils encaissent trop de coups.

I.5. Interfaces

Menu

Un menu permet de changer les paramètres, de consulter la carte ou de quitter le jeu.

Paramètres

Il est possible de modifier le volume du jeu et les touches du clavier.

Inventaire

L'inventaire est situé en haut à gauche de l'écran, via une icône représentant chaque objet à obtenir, avec la quantité disponible et le nombre nécessaire pour terminer le jeu (construire le radeau).

Barre d'outils

La barre d'outils est située en bas de l'écran, permettant de sélectionner l'outil à utiliser.

Carte

Le joueur se déplace sur la carte, et la caméra suit en temps réel. Une mini-carte est présente en bas à gauche de l'écran.

Boîte de dialogue

Une boîte de dialogue affiche le but du jeu, les différentes étapes et les règles.

I.6. Spécifications techniques

Le jeu est développé en Kotlin, en utilisant la bibliothèque JavaFX Game Library (FXGL) pour faciliter le développement et fournir une interface graphique. Il doit être compatible avec Windows, Mac et Linux.

I.7. Contraintes

Le projet doit être rendu avant le 20 octobre 2024.

II. Document d'analyse

II.1. Cas d'utilisation

Voici le diagramme de cas d'utilisation pour la partie joueur du jeu **Escape The Chicken Island**:

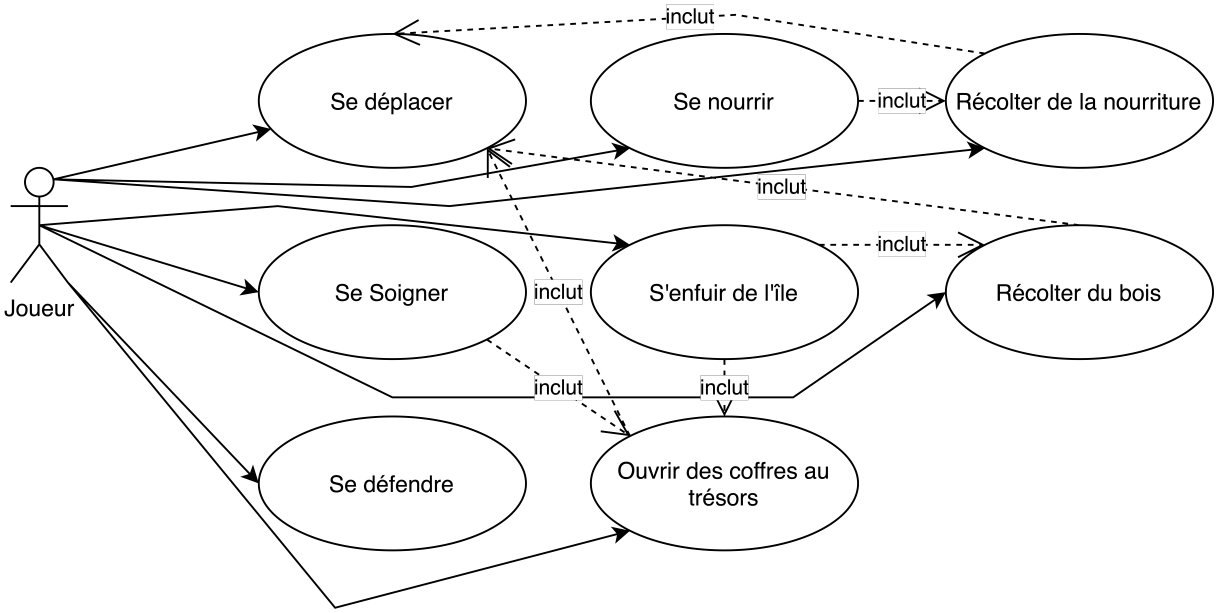


Fig. 1. – Diagramme de cas d'utilisation joueur

Voici le diagramme pour la partie système:

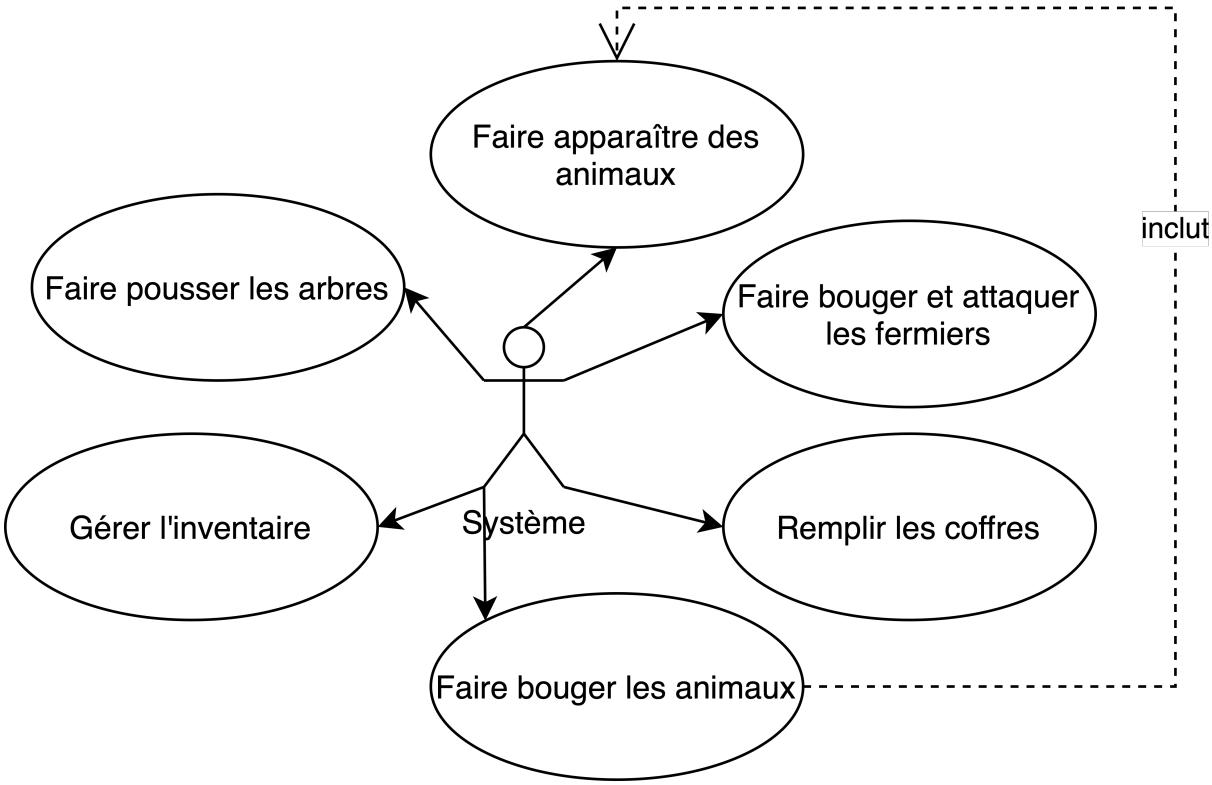


Fig. 2. – Diagramme de cas d'utilisation système

Récolter du bois

Ci-dessous la description du cas d'utilisation, **Récolter du bois** :

1. **Objectif :** Pour construire le radeau, le joueur a besoin de bois.

1.1 **Équiper la hache :** Si le joueur n'a pas la hache équipée, il doit l'équiper.

1.1.1 **Navigation entre les outils :** Le joueur peut défiler avec la molette de la souris ou appuyer sur **Tab** ou **Shift + Tab** pour naviguer à travers les outils situés en bas de l'écran.

1.2 **S'approcher d'un arbre coupable :** Une fois la hache équipée, le joueur s'approche d'un arbre susceptible d'être coupé.

1.3 **Couper l'arbre :** Le joueur appuie sur **Space** pour couper l'arbre et récolter du bois. Le bois collecté est ajouté à la quantité de bois affichée en haut à gauche de l'écran.

Voici le diagramme de séquence système associé:

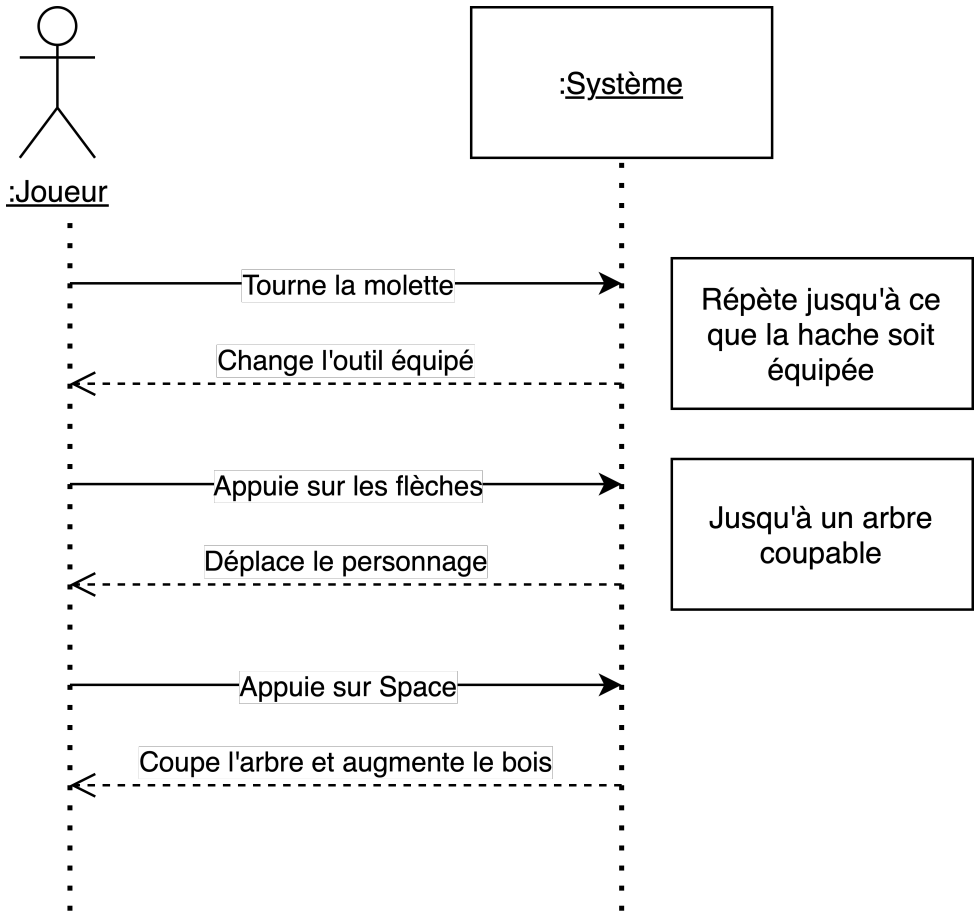


Fig. 3. – Diagramme de séquence récolte bois

Récolter de la nourriture

Ci-dessous la description du cas d'utilisation, **Récolter de la nourriture** :

1. **Objectif :** Pour survivre, le joueur a besoin de nourriture.

1.1 **Équiper l'épée :** Si le joueur n'a pas l'épée équipée, il doit l'équiper.

1.1.1 **Navigation entre les outils :** Le joueur peut défiler avec la molette de la souris ou appuyer sur **Tab** ou **Shift + Tab** pour naviguer à travers les outils situés en bas de l'écran.

1.2 **S'approcher d'un animal :** Une fois l'épée équipée, le joueur s'approche d'un animal.

1.3 **Tuer l'animal :** Le joueur appuie sur **Space** pour tuer l'animal et récolter de la nourriture. La nourriture collectée est ajoutée à la quantité de nourriture affichée en bas de l'écran.

Voici le diagramme de séquence système associé:

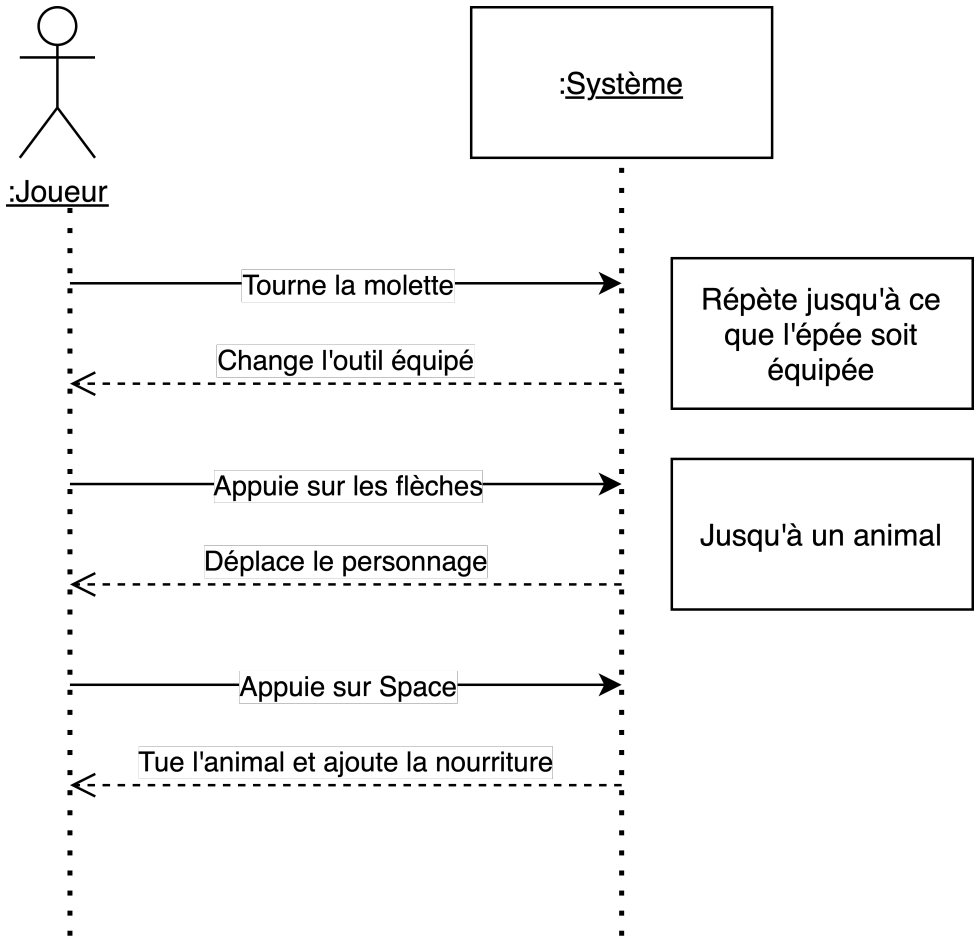


Fig. 4. – Diagramme de séquence récolte nourriture

Se battre contre un fermier

Ci-dessous la description du cas d'utilisation, **Se battre contre un fermier** :

1. **Objectif :** Pour survivre, le joueur doit se battre contre un fermier.

1.1 **Équiper l'épée :** Si le joueur n'a pas l'épée équipée, il doit l'équiper.

1.1.1 **Navigation entre les outils :** Le joueur peut défiler avec la molette de la souris ou appuyer sur **Tab** ou **Shift + Tab** pour naviguer à travers les outils situés en bas de l'écran.

1.2 **S'approcher d'un fermier :** Une fois l'épée équipée, le joueur s'approche d'un fermier.

1.3 **Se battre contre le fermier :** Le joueur appuie sur **Space** autant de fois que nécessaire pour mettre K.O. le fermier.

Voici le diagramme de séquence système associé:

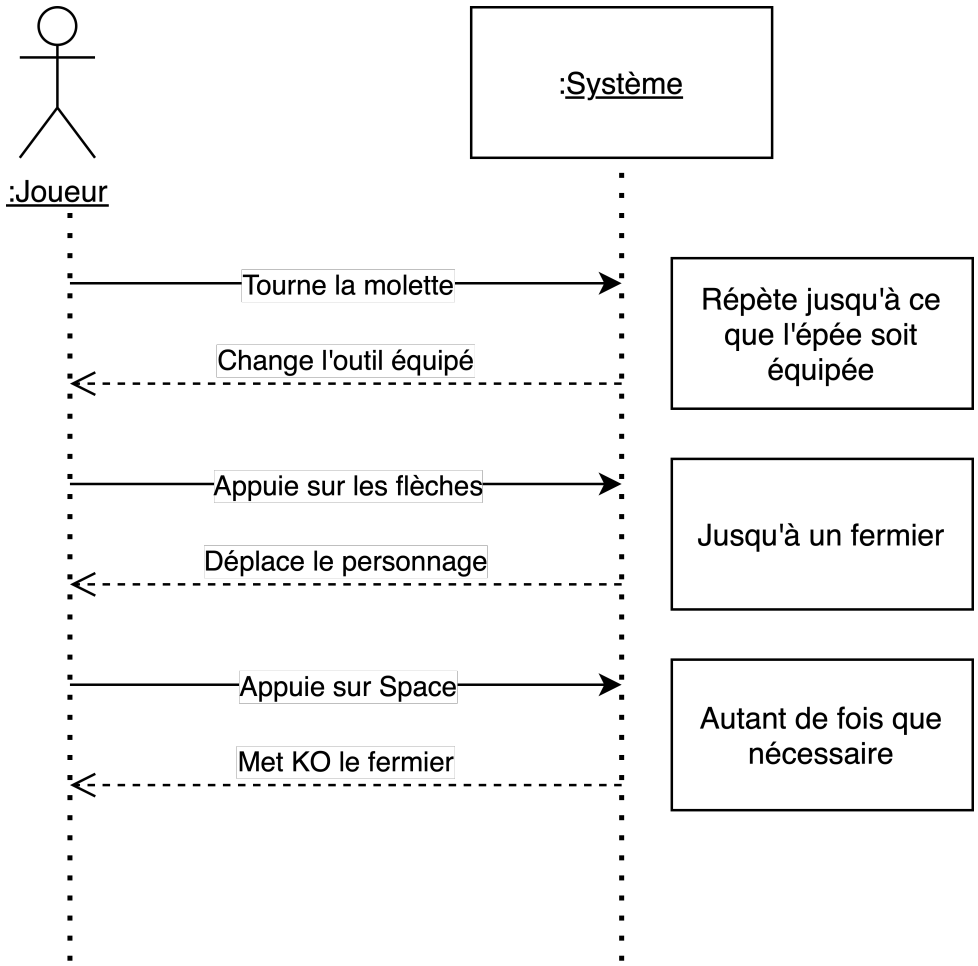


Fig. 5. – Diagramme de séquence combat fermier

Se nourrir

Ci-dessous la description du cas d'utilisation, **Se nourrir** :

1. **Objectif :** Pour survivre, le joueur a besoin de se nourrir.

1.1 **Récolter de la nourriture :** Si le joueur n'a pas de nourriture, il doit en récolter.

1.2 **Sélectionner la nourriture :** Lorsque le joueur dispose de nourriture, il doit la sélectionner. Pour cela, il peut défiler avec la molette de la souris ou appuyer sur **Tab** ou **Shift + Tab** pour naviguer à travers les outils situés en bas de l'écran.

1.3 **Consommer la nourriture :** Une fois la nourriture équipée, le joueur appuie sur **Space** pour la consommer et restaurer sa jauge de faim.

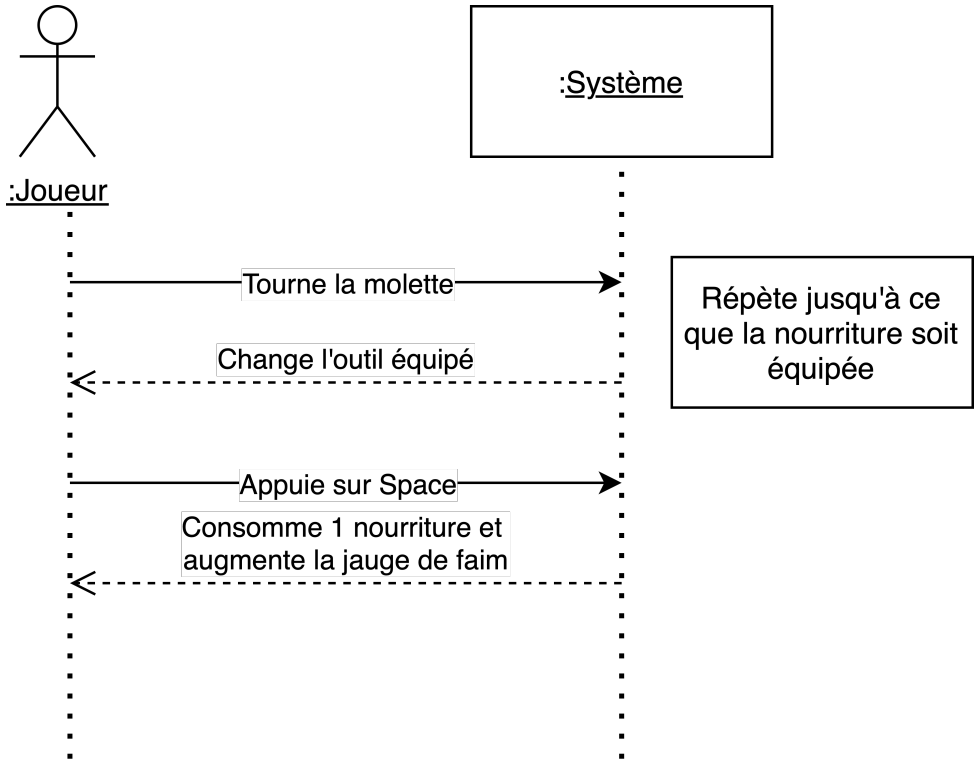


Fig. 6. – Diagramme de séquence consommation nourriture

Se soigner

Ci-dessous la description du cas d'utilisation, **Se soigner** :

1. **Objectif :** Pour survivre, le joueur a besoin de se soigner.

1.1 **Récolter une potion :** Si le joueur n'a pas de potion, il doit en récolter dans un coffre.

1.2 **Sélectionner la potion :** Lorsque le joueur dispose d'une potion, il doit la sélectionner. Pour cela, il peut défiler avec la molette de la souris ou appuyer sur **Tab** ou **Shift + Tab** pour naviguer à travers les outils situés en bas de l'écran.

1.3 **Consommer la potion :** Une fois la potion équipée, le joueur appuie sur **Space** pour la consommer et restaurer sa jauge de vie.

Voici le diagramme de séquence système associé:

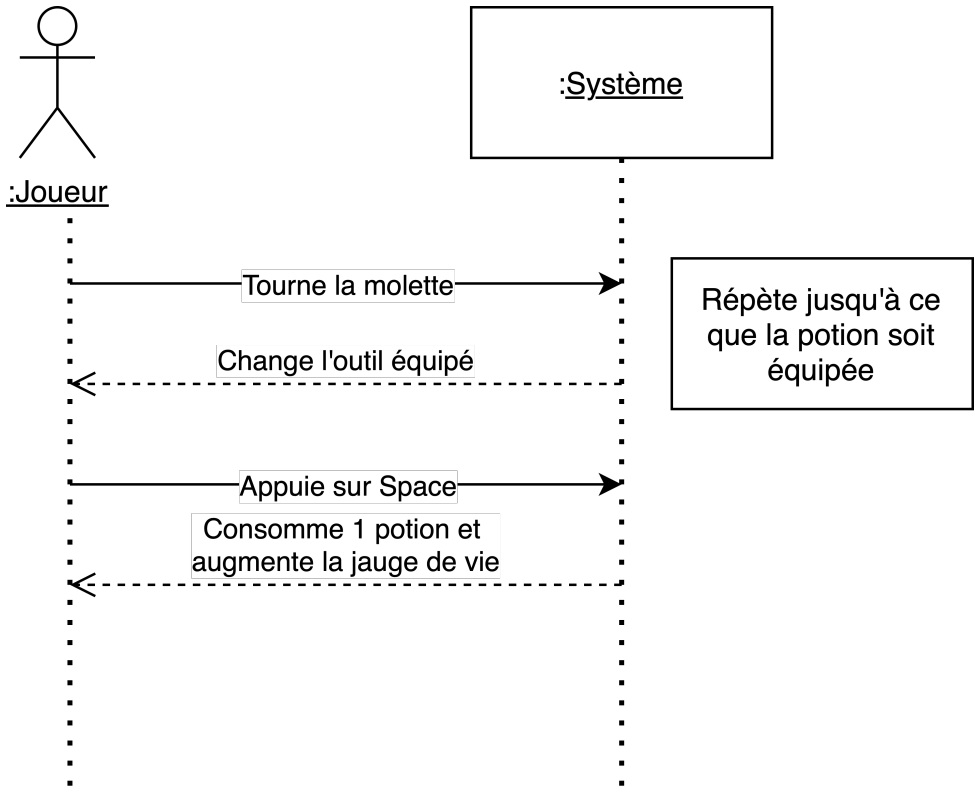


Fig. 7. – Diagramme de séquence consommation potion

Ouvrir un coffre au trésor

Ci-dessous la description du cas d'utilisation, **Ouvrir un coffre au trésor** :

1. **Objectif :** Pour survivre et s'échapper, le joueur a besoin des objets disponibles dans les coffres au trésor.

1.1 **Équiper la main :** Si le joueur n'a pas la main équipée, il doit l'équiper.

1.1.1 **Navigation entre les outils :** Le joueur peut défiler avec la molette de la souris ou appuyer sur **Tab** ou **Shift + Tab** pour naviguer à travers les outils situés en bas de l'écran.

1.2 **S'approcher d'un coffre au trésor :** Une fois la main équipée, le joueur s'approche d'un coffre au trésor.

1.3 **Ouvrir le coffre :** Le joueur appuie sur **Space** pour ouvrir le coffre et récolter un objet. L'objet collecté est ajouté à l'inventaire s'il s'agit de matériaux, ou à la barre d'outils s'il s'agit d'une potion de soin.

Voici le diagramme de séquence système associé:

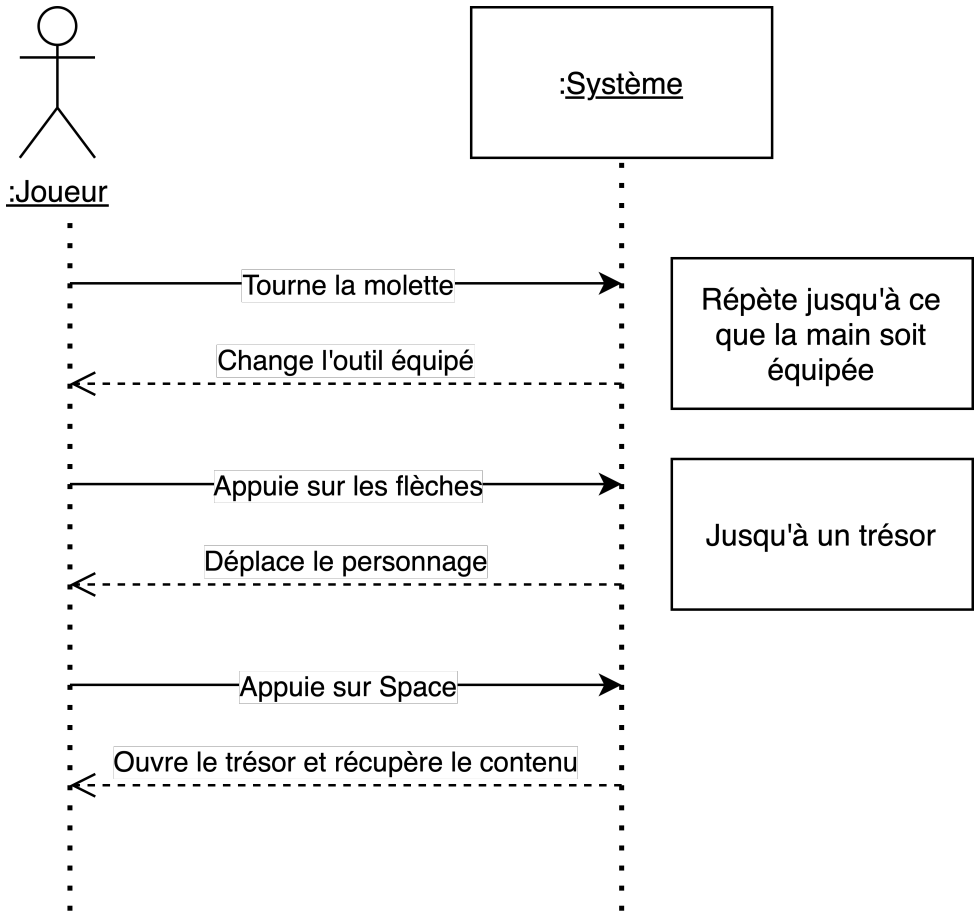


Fig. 8. – Diagramme de séquence récolte trésor

Finir le jeu

Ci-dessous la description du cas d'utilisation, **Finir le jeu** :

1. **Objectif :** Une fois les matériaux accumulés, le joueur souhaite pouvoir finir le jeu.

1.1 **Se déplacer :** Le joueur se déplace à l'aide des flèches vers le quai des îles.

1.2 **Construire le radeau :** Le joueur interagit en appuyant sur **Space** pour construire le radeau et s'échapper.

Voici le diagramme de séquence système associé:

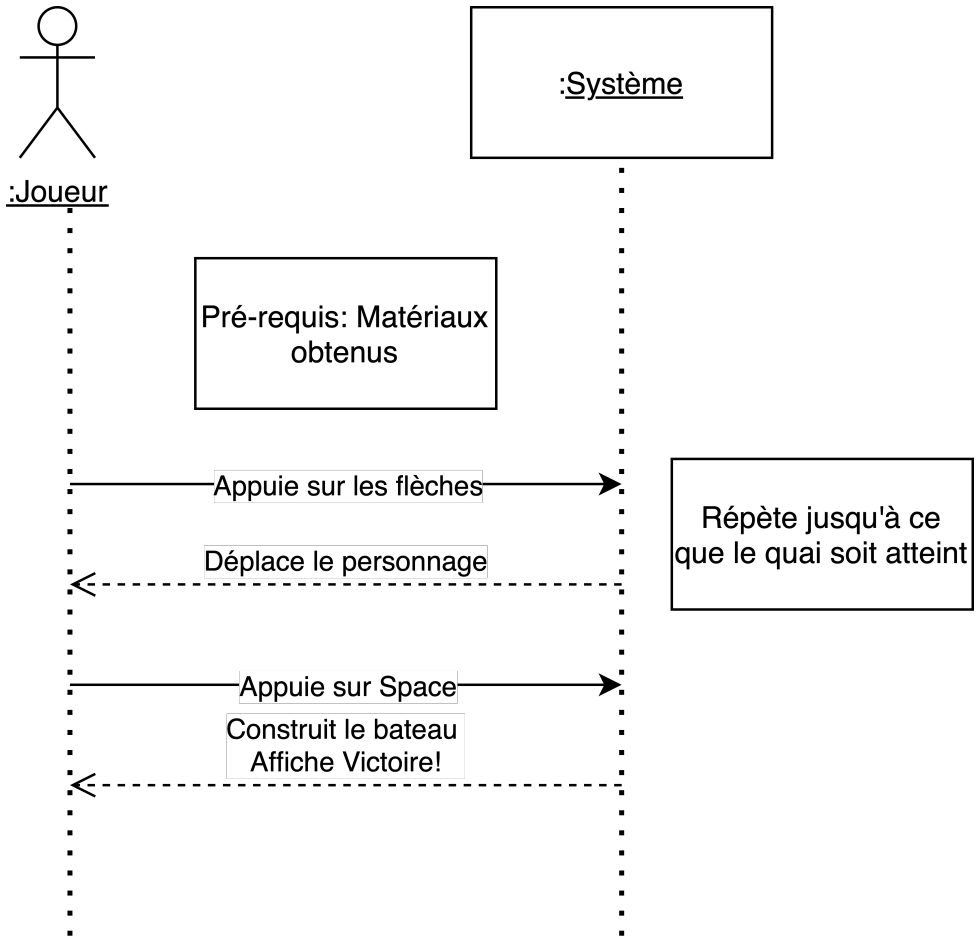


Fig. 9. – Diagramme de séquence finir le jeu

II.2. Diagramme de classes d'analyse

Afin de démarrer le code du jeu, nous avons réalisé un diagramme de classes d'analyse. Ce diagramme permet de définir les classes et les relations entre elles. Il est important de noter que ce diagramme est une première version et qu'il est susceptible d'être différent de la version finale présentée dans le document de conception.

Lors de cette phase, nous nous sommes inspirés des patrons de conception **Singleton** et **Fabrique** afin de créer des classes qui permettent de gérer les objets du jeu de manière efficace. En effet, le joueur suit le patron du **Singleton** pour être unique, tandis que les objets du jeu suivent le patron de la **Fabrique** pour être créés de manière dynamique.

Voici le diagramme de classes d'analyse :

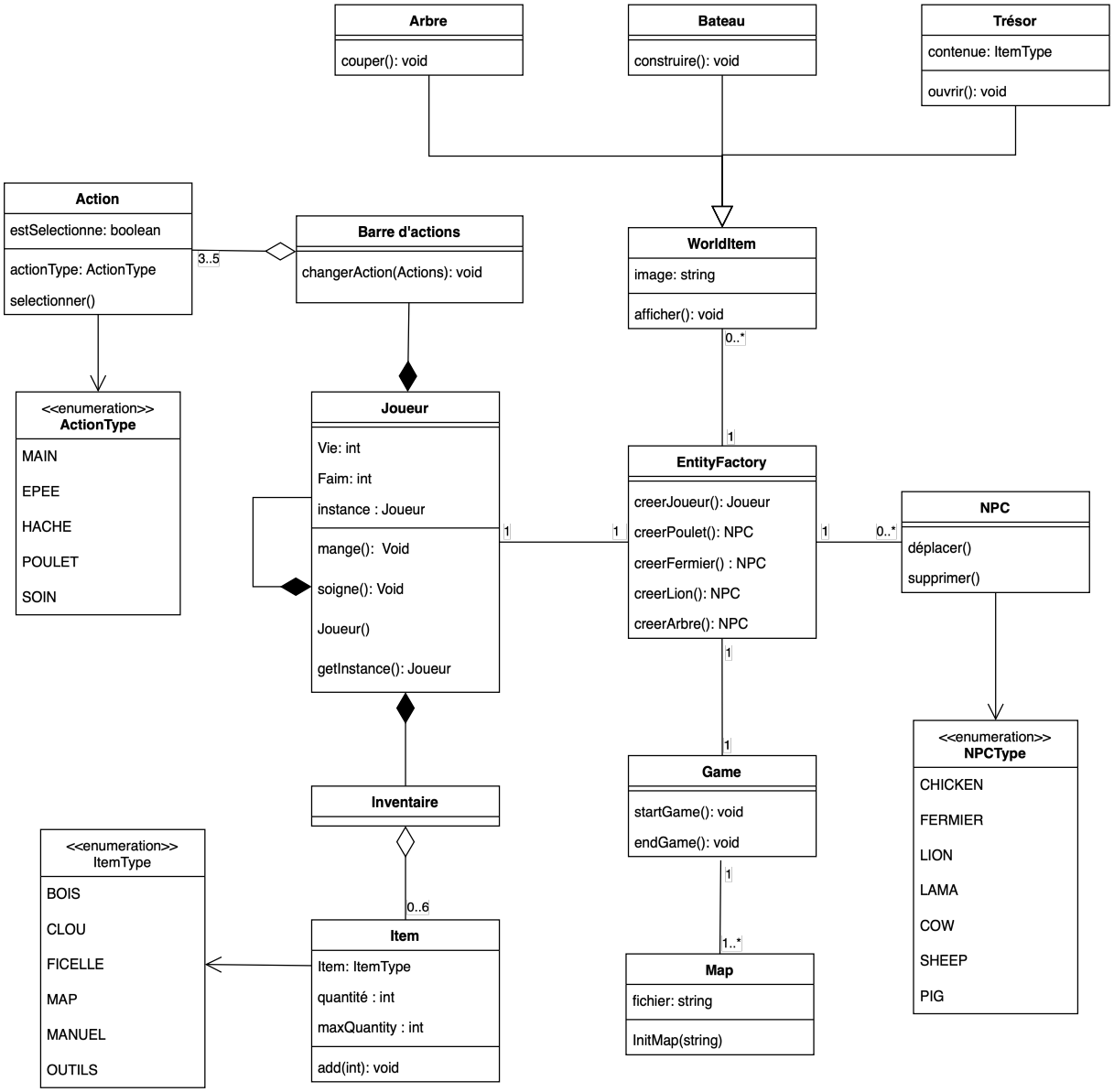


Fig. 10. – Diagramme de classe d'analyse

Vous pouvez consulter le diagramme dans le dossier **Doc/Diagrammes/Diagramme_classe_analyse.png**.

III. Document de conception

III.1. Architecture logicielle

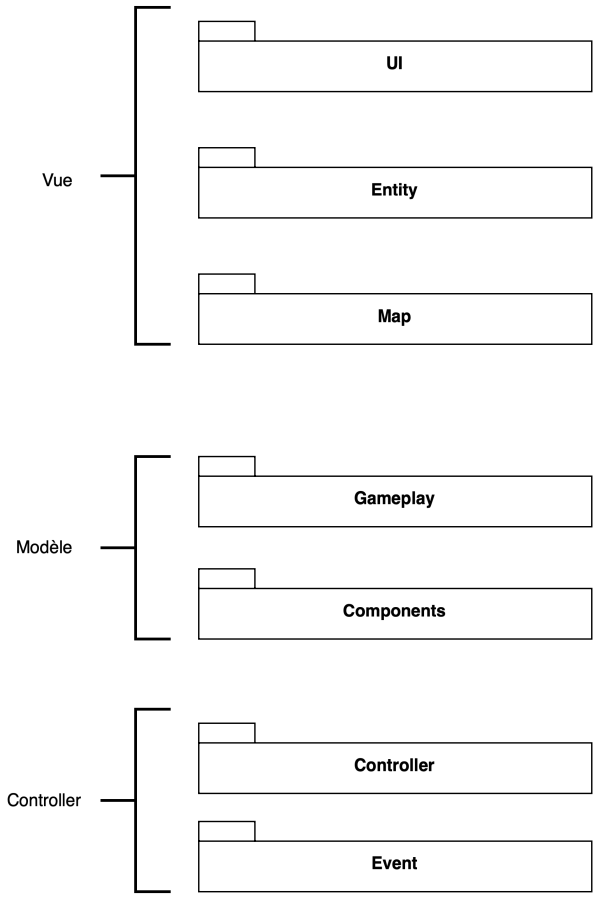


Fig. 11. – Diagramme d'architecture

Concernant l'architecture de notre jeu, nous avons choisi de suivre un modèle MVC tout en l'adaptant légèrement afin d'utiliser FXGL et Kotlin efficacement, comme vous pouvez le voir ci-dessus. Cela permet de différencier les couches de notre application afin de la rendre plus modulaire et plus facile à maintenir. Le but est donc de réunir les classes qui ont des responsabilités similaires et de les regrouper dans des packages distincts.

Nous avons donc la vue, qui regroupe la carte, la génération des objets sur cette carte, les animations, les sprites, l'inventaire et la barre d'outils.

Le contrôleur gère les interactions entre le joueur et la vue, ainsi que les événements.

Le modèle contient les classes qui représentent le gameplay du jeu et les différents composants de notre jeu. Ici les composants ou **components** sont des classes qui servent de modèle. On a donc un **component** pour chaque entité du jeu, comme le joueur, les animaux, les coffres, etc.

III.2. Description de l'incrément choisit

III.3. Premier incrément

Nous avons décidé, comme premier incrément, de fournir une partie du jeu déjà jouable.

Nous avons réalisé une carte où l'on peut se déplacer avec un personnage afin de survivre. Nous avons implémenté un inventaire qui permet de posséder des matériaux et des outils. Les actions réalisables par le personnage sont les suivantes :

- **Ouvrir un coffre**
- **Couper un arbre pour obtenir du bois**
- **Tuer un poulet pour obtenir de la nourriture**
- **Manger**
- **Se soigner**

La carte contient quatre îles, sans conditions d'accès pour l'instant, avec des zones où l'on peut marcher, passer devant des objets, et passer derrière des objets.

L'inventaire prend la forme d'une check-list, où l'on peut voir si l'on a ou non assez de matériaux pour quitter l'île. Le bois est disponible en coupant des arbres qui repoussent avec le temps, tandis que les autres matériaux se trouvent dans les coffres qui se remplissent également avec le temps.

Une fois les matériaux obtenus, on peut quitter l'île en se rendant au radeau, situé au bout du quai de l'île désertique, et en interagissant avec celui-ci.

Pour l'instant, il n'y a pas d'ennemis, à l'exception de la faim, qui tire sur le joueur et menace de lui faire perdre de la vie si la jauge de faim est vide.

III.4. Conception détaillée

Diagramme de classe logicielle

Pour faire suite au diagramme de classe d'analyse, nous avons pu étayé nos classes et leurs relations. Effectivement, pour respecter l'architecture décrite plus haut tout en utilisant efficacement FXGL et Kotlin, nous avons dû adapter notre diagramme de classe. Nous avons également adapté légèrement l'architecture afin de rendre un premier incrément complet, nous sommes conscient que l'architecture peut être amélioré pour le jeu final.

Voici le diagramme de classe logicielle modifié durant nos développements :

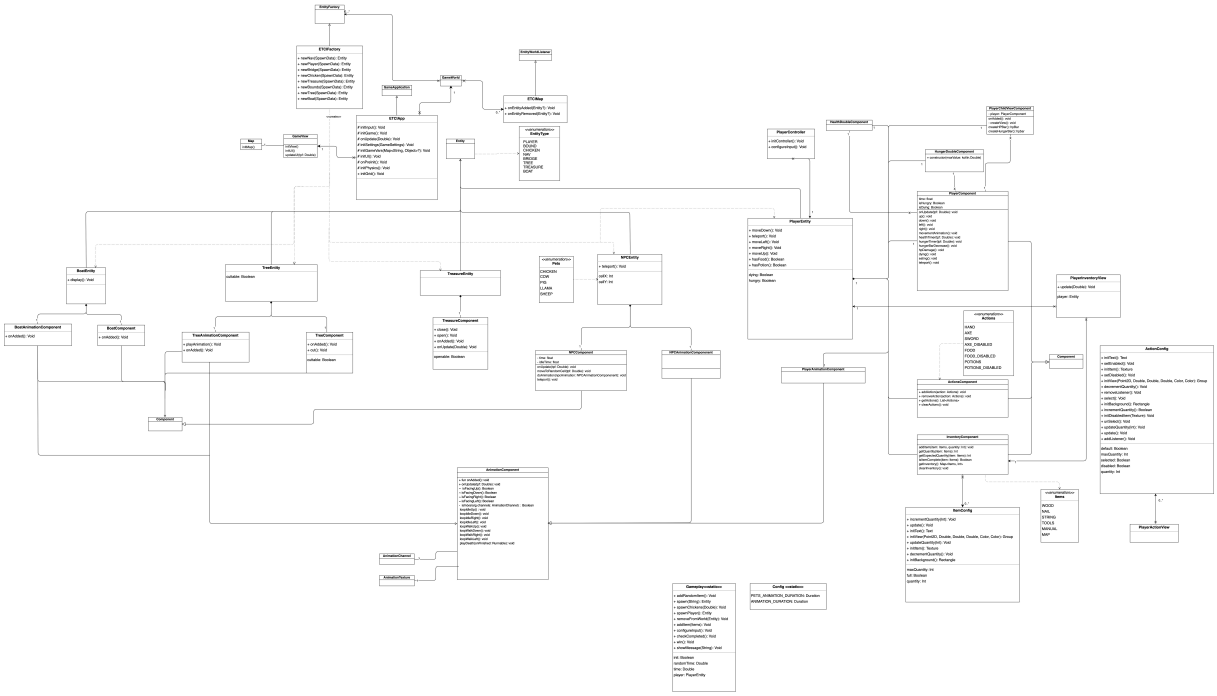


Fig. 12. – Diagramme de classe logicielle

Le diagramme de classe logicielle étant très complet, il est quasiment illisible sur tirage A4. Nous vous conseillons de le consulter dans le dossier **Doc/Diagrammes/UML.pdf**. (Et non png)

Diagrammes de séquence

Pour ce qui a été conçu, on peut reprendre les diagrammes de la partie analyse qui permettaient d'implémenter les fonctionnalités générales du jeu. (Excepté celui du combat contre un fermier, car non implémenté). En effet, ces fonctionnalités ayant été implémentées, les diagrammes de séquence associés sont toujours d'actualité.

Pour avoir une idée plus précise de l'implémentation, voici un exemple de diagramme de séquence plus précis pour l'ouverture d'un coffre au trésor :

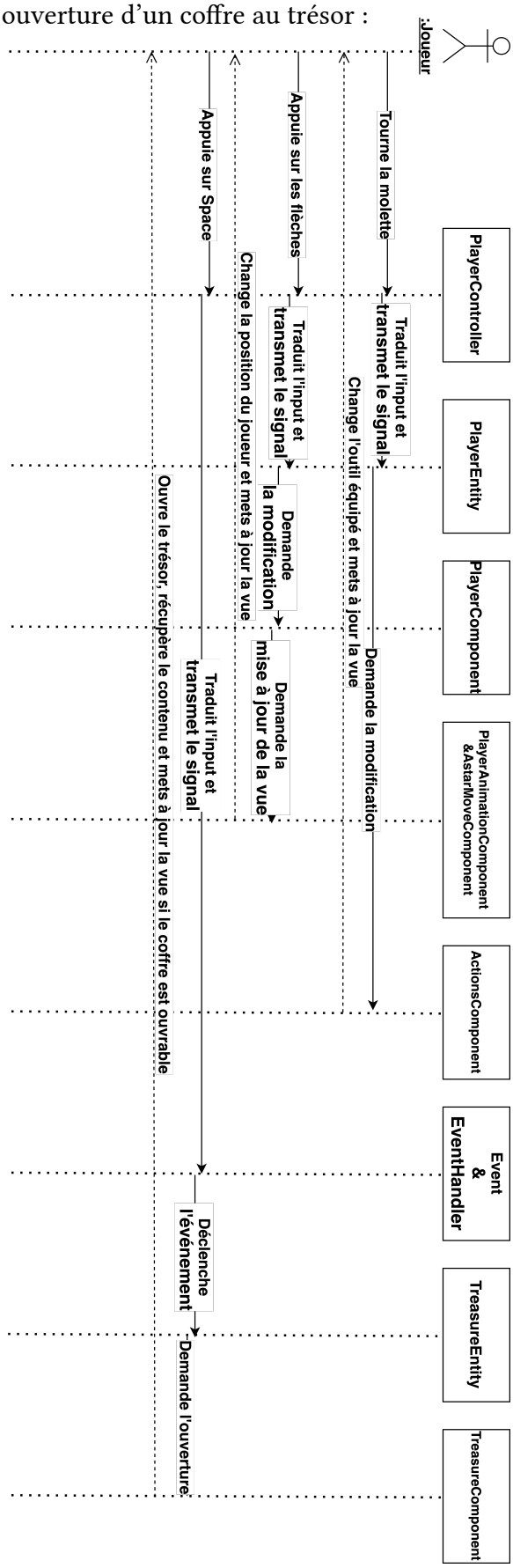


Fig. 13. – Diagramme de séquence récolte trésor

Nous voyons bien ici que l'architecture du jeu suit la logique décrite précédemment. En effet, une action de l'utilisateur passe par un contrôleur, puis une entité, puis un composant. (À l'exception près des événements gérés différemment et des fonctionnalités de FXGL qui sont gérées par le framework).

IV. Manuel utilisateur

La partie compilation du projet et lancement du jeu est réalisable en suivant les instructions du **A_LIRE.txt** fourni avec le projet.

Concernant le jeu en lui-même, voici les commandes utilisables :

- Les flèches directionnelles pour se déplacer
- **Tab** ou **Shift + Tab** ou défilement de la molette pour naviguer à travers les outils en bas de l'écran (on peut aussi cliquer sur les outils)
- **Space** pour interagir avec l'outil équipé
- **T** pour se téléporter aléatoirement sur la carte (fonction pour faciliter le développement)
- **Échap** pour ouvrir le menu géré grâce à FXGL, qui propose pour l'instant de quitter le jeu ou de changer des options comme : le volume (pas de son pour l'instant), les touches, et la langue du menu.

Quelques astuces :

- Les arbres de l'île en haut à gauche sont les seuls coupables et repoussent avec le temps.
- Les coffres se remplissent selon une certaine probabilité et contiennent un objet aléatoire pour construire le radeau ou une potion.
- De base, quasiment tous les matériaux sont déjà possédés par le joueur afin de tester le jeu plus facilement.
- La jauge de faim descend ; attention à manger pour ne pas mourir.
- Moins il y a de poulets sur les îles, plus ils mettent de temps à se reproduire.
- Pensez à utiliser le bon outil selon l'action désirée. (Pour le radeau, il s'agit de la main).
- Il n'y a qu'une carte disponible et on ne peut pas rentrer dans les grottes ou maisons.

V. Bilan sur les outils de modélisation

V.1. Draw.io

Draw.io est un outil de modélisation en ligne qui permet de réaliser des diagrammes. Il est très complet et facile d'utilisation. Il permet de partager les diagrammes avec un lien, de les exporter en image, PDF, XML, etc. De cette façon, il est simple de travailler à plusieurs sur un diagramme. Il est très pratique pour réaliser des diagrammes simples et rapides.

Cependant, il est moins adapté pour des diagrammes plus complexes et plus détaillés. En effet, il est difficile de gérer les relations entre les différents éléments d'un diagramme et, même si on peut créer facilement des zones de texte, il est plus simple d'utiliser un outil qui prévoit déjà les zones pour l'écriture d'attributs et de méthodes. C'est pourquoi nous l'avons utilisé pour les diagrammes de séquence uniquement.

V.2. Visual Paradigm

Visual Paradigm est un outil de modélisation complet et très puissant. Il permet de réaliser des diagrammes de classes, de séquence et d'états-transitions. Il est souvent utilisé pour concevoir, modéliser et documenter des systèmes logiciels complexes. Certains membres du groupe étaient déjà familiers avec cet outil, c'est donc pour cela que, malgré sa complexité, nous l'avons utilisé.

Il s'intègre bien avec des environnements de développement populaires comme IntelliJ IDEA et Visual Studio (IDE que nous avons utilisés), facilitant ainsi le passage entre la conception et l'implémentation. Visual Paradigm propose des fonctionnalités pour le travail collaboratif en temps réel, permettant aux équipes de travailler ensemble sur le même projet. Pour ce projet, la version gratuite a largement suffi.